

Technical Specifications: Smart Analysis Framework

1. System Architecture Overview

The framework acts as a high-level orchestration layer within the Canvas environment. It processes document selections through a template-driven pipeline using a standardized JSON contract.

1.1 High-Level Stack

- **Frontend:** React 18+ with **React Flow** for the pipeline editor (Studio) and Tailwind CSS for the execution UI.
- **Orchestrator:** Backend logic (Node.js/Python) responsible for parsing the Template JSON and managing the execution lifecycle.
- **LLM Integration:** Agnostic. Interfaces with existing multi-provider implementation (Cloud/Local).
- **Persistence:** **RDBMS** for storing templates, personas, and framework rules.

2. Generic Data Schema (The "Contract")

Le schéma doit être agnostique au domaine métier. Voici la structure normalisée qui sera produite par le Studio et consommée par l'Engine.

2.1 The Smart Template Object

```
{
  "template_id": "uuid",
  "metadata": {
    "name": "string",
    "version": "string",
    "category": "Strategic | Compliance | Technical | Legal",
    "description": "string"
  },
  "execution_constraints": {
    "input_mode": "single | multi",
    "min_assets": "number",
    "max_assets": "number"
  },
  "user_parameters": [
    {
```

```

    "id": "string",
    "label": "string",
    "type": "text | number | date | time | list_single | list_multi",
    "description": "string (mandatory for AI grounding)",
    "options": ["string"],
    "required": "boolean"
  }
],
"pipeline": [
  {
    "step": 1,
    "module": "data_extractor",
    "params": {
      "target_sections": ["string"],
      "target_entities": ["string"],
      "objectives": "string"
    }
  },
  {
    "step": 2,
    "module": "ai_agent",
    "params": {
      "persona_id": "string",
      "framework_id": "string",
      "logic_depth": "fast | chain_of_thought | deep_audit",
      "thesaurus_id": "string (optional)"
    }
  },
  {
    "step": 3,
    "module": "output_formatter",
    "params": {
      "synthesis_category": "Text | Tables | Pictures | Diagrams | Combination",
      "rendering_type": "string (from Master Reference)",
      "file_formats": ["md", "csv", "svg"]
    }
  }
]
}

```

3. The Analysis Lifecycle

3.1 Step 1: Pre-processing (Context Optimization)

L'Engine utilise la couche de vectorisation existante de l'application pour extraire uniquement les `target_sections` définies. Cela réduit le bruit et optimise la fenêtre de contexte du LLM.

3.2 Step 2: Individual Analysis (Map Phase)

Pour chaque document sélectionné, l'Engine génère des **Evidence Objects**.

```
{  
  "id": "uuid",  
  "snippet": "raw_text_source",  
  "asset_id": "canvas_id",  
  "page": "number",  
  "confidence": "float"  
}
```

3.3 Step 3: Synthesis (Reduce Phase)

L'Engine agrège tous les Evidence Objects et les injecte dans le `ai_agent` avec les `user_parameters` saisis par l'utilisateur final. L'IA produit alors la synthèse finale en respectant le `framework_id` et la `persona_id`.

4. UI/UX & Visual Rendering

- **Studio:** Utilise React Flow pour manipuler le tableau pipeline comme un graphe de noeuds.
- **Mermaid & SVG:** Les modules `output_formatter` génèrent du code (Mermaid) ou des chemins de données (Tables) rendus dynamiquement par le Frontend.
- **Traceability:** Chaque affirmation dans le rapport final doit porter l'ID d'un Evidence Object pour permettre le "Source Highlighting".

5. Execution Guardrails

- **Concurrency Mode:** Configurable par template ou globalement (Séquentiel pour LLM locaux, Parallèle pour Cloud).
- **Temperature:** Fixée à 0.0 pour l'extraction et 0.1 pour la synthèse afin de garantir la reproductibilité.
- **Validation:** Étape de post-traitement optionnelle pour valider la conformité terminologique avec le `thesaurus_id`.

End of Technical Specifications